

Examen Procesadores de Lenguajes

07/07/25

-Enunciados:

TEORÍA

1. *Estructura de un compilador:* (2 puntos)

- Fases
- Pasos

2. *Tipos de reconocimiento de palabras claves en el análisis léxico:* (1 punto)

- Descripción
- Ejemplos

EJERCICIOS

3. *Dada la siguiente expresión regular: $a(b + c)^*a$* (2 puntos)

a) *Utiliza el Algoritmo de Thompson para construir un AFN equivalente a la expresión regular.*

b) *Utiliza el Algoritmo de Construcción de Subconjuntos para obtener un AFD equivalente al AFN obtenido en el apartado "a".*

c) *Minimiza, si es posible, el AFD obtenido en el apartado anterior.*

d) *Comprueba si el último autómatas obtenido reconoce la siguiente cadena:*

a b c a

4. *Análisis descendente predictivo* (2 puntos)

- *Considérese la siguiente gramática de contexto libre:*

P = {

1. $S \rightarrow SD$

2. $S \rightarrow D$

3. $D \rightarrow L : TV ;$

4. $L \rightarrow id$

5. $L \rightarrow L , id$

6. $T \rightarrow real$

7. $T \rightarrow entero$

8. $V \rightarrow \epsilon$

9. $V \rightarrow = \text{número}$

}

- Esta gramática permite generar declaraciones de variables, como por ejemplo:
 $a, b, c: \text{entero} = 1;$
 $x, y: \text{real};$
 donde b, d, x, y son identificadores y 1 es un número.

- Elimina la recursividad inmediata por la izquierda y factoriza la gramática por la izquierda.
- A partir de la gramática obtenida en el apartado "a":
 - Construye los conjuntos "**primero**" y "**siguiente**" de los símbolos no terminales.
 - Construye la tabla de análisis **descendente predictivo**.
 - Utiliza el método recuperación de errores de "**nivel de frase**" para completar la tabla predictiva.
 - Utiliza la tabla predictiva para realizar un análisis descendente **no recursivo** de la siguiente declaración errónea: $a, , c: \text{entero } 1$

5. Análisis sintáctico ascendente SLR

(2 puntos)

- Considérese la siguiente gramática de contexto libre:

$P = \{$

- $S \rightarrow SD$
- $S \rightarrow \varepsilon$
- $D \rightarrow \text{struct id } \{ L \};$
- $L \rightarrow LI$
- $L \rightarrow I$
- $I \rightarrow T \text{ id};$
- $T \rightarrow \text{float}$
- $T \rightarrow \text{int}$

$\}$

- Esta gramática permite generar declaraciones de estructuras como la siguiente:
 $\text{struct PUNTO } \{$
 $\quad \text{float } x;$
 $\quad \text{float } y;$
 $\};$
 donde x e y son identificadores.

- Construye los conjuntos "**primero**" y "**siguiente**" de los símbolos no terminales.
- Construye la colección canónica de $LR(0)$ – **elementos** del análisis SLR.
- Dibuja el **autómata** que reconoce los prefijos viables.
- Construye la tabla de análisis sintáctico SLR: partes **acción** e **ir_a**.

e) Utiliza el método recuperación de errores de “nivel de frase” para completar la tabla de análisis SLR.

f) Utiliza la tabla de análisis SLR y el método de nivel de frase para analizar la siguiente declaración errónea: **struct PUNTO float ; y }**

6. Diseño de una gramática de contexto libre (1 punto)

· Diseña una gramática de contexto libre que permita generar palabras del siguiente lenguaje formal:

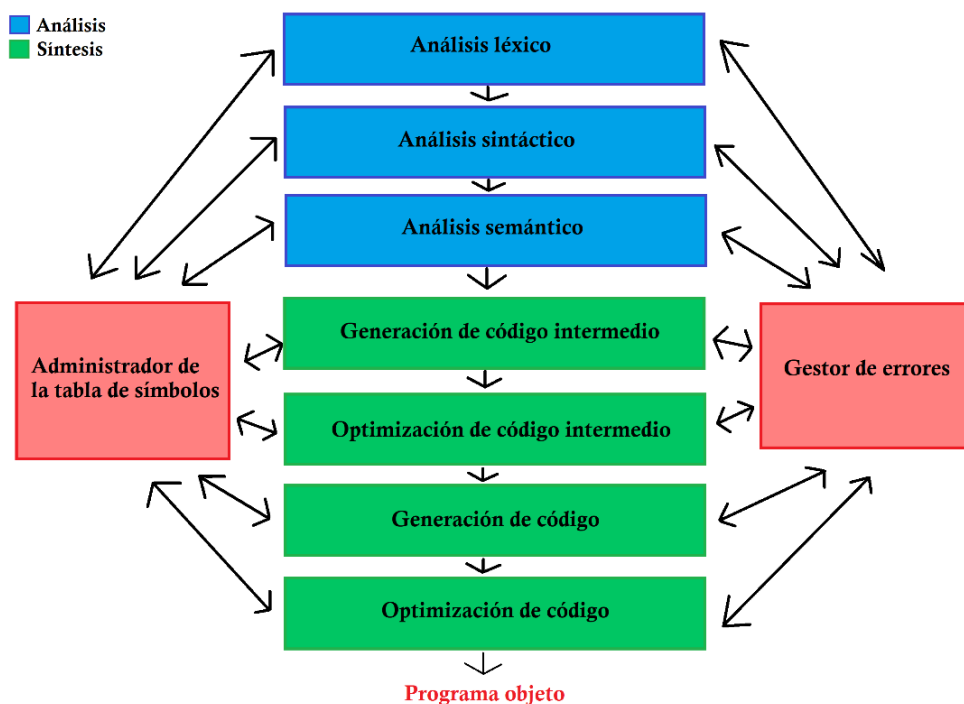
$$L = \{a^i b^{2j} c^j d^{2i} \mid i > 0, j > 1\}$$

Aclaración: Las respuestas de teoría son totalmente íntegras a lo que se debe responder (excepto la representación gráfica del ejercicio 1), pero en los ejercicios se incluye una gran cantidad de texto para llegar a entender cómo realizar los ejercicios sin haber realizado ningún otro anteriormente, por lo que la mayoría de texto e ilustraciones en estos ejercicios no son necesarias.

-Soluciones:

Ejercicio 1:

Las fases de un compilador se dividen en **análisis** y **síntesis**. El **análisis** se encarga de comprobar que el programa fuente está bien escrito y la **síntesis** se ocupa de la generación del código ejecutable. También consta de **componentes auxiliares** durante este proceso como el administrador de la tabla de símbolos y el gestor de errores a los que **se accede en cada uno de los pasos** del compilador. Durante el proceso de compilación **se empieza con el análisis**, cuyos pasos constan del **análisis léxico** (que consiste en leer el programa fuente y obtener **tokens** o **componentes léxicos**), el **análisis sintáctico** (en el que se utilizan reglas gramaticales del lenguaje fuente y los componentes léxicos para generar el **árbol sintáctico**) y el **análisis semántico** (que comprueba si el significado de las sentencias es correcto). Y posterior al análisis se continúa con **la síntesis** cuyos pasos son **la generación y optimización de código intermedio** (que es una representación intermedia del código usando definiciones dirigidas por la sintaxis o esquemas de traducción el cual es un paso opcional pero recomendable) y **la generación y optimización de código** (donde se genera el programa objeto más eficiente en tiempo y espacio el cual debe de preservar el significado del código). Una representación gráfica del proceso es la siguiente:



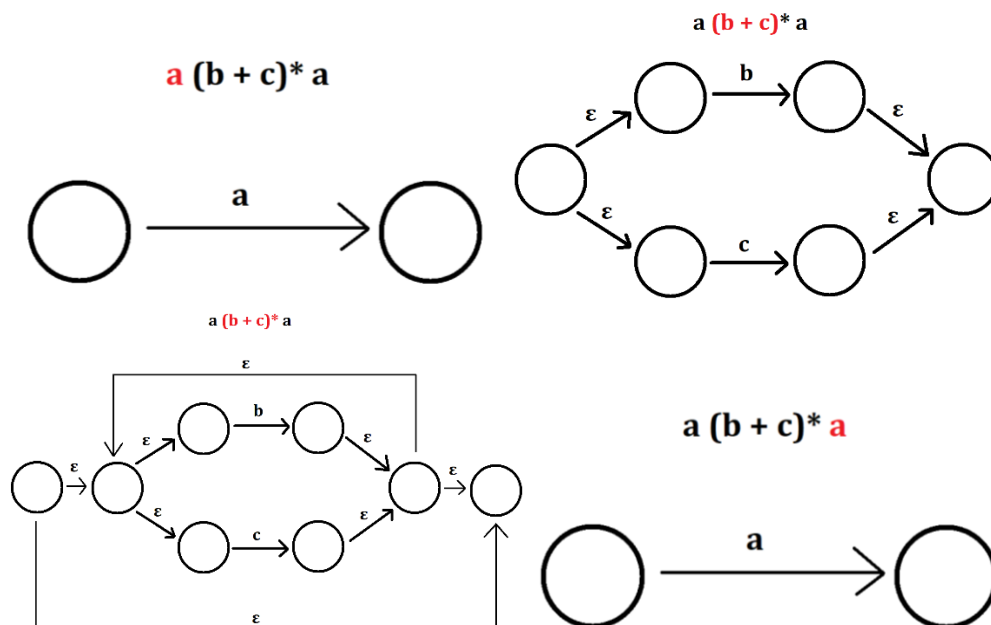
Ejercicio 2:

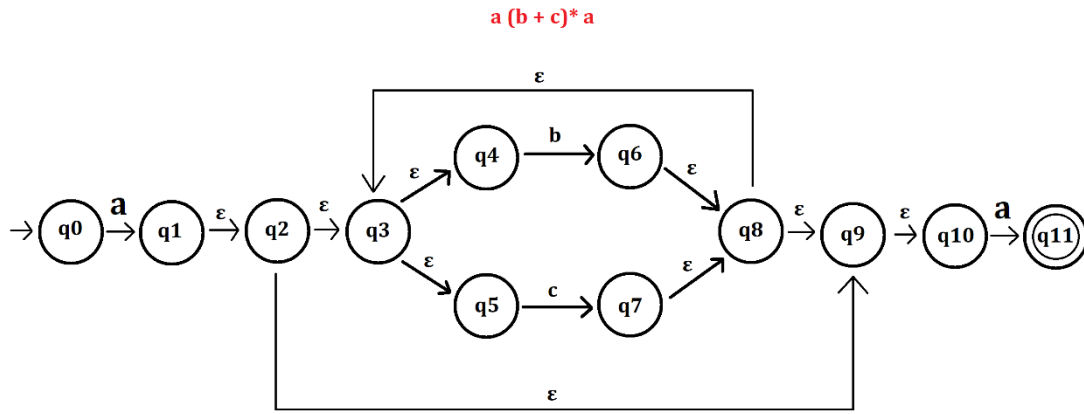
Hay dos maneras principales de poder reconocer los componentes léxicos durante el análisis léxico: el **implícito** y el **explícito**. El **reconocimiento implícito** es aquél que tiene los nombres y componentes léxicos **pre-instalados** en la tabla de símbolos. Este reconocimiento es más sencillo, pero necesita más tiempo y espacio. Un ejemplo de este tipo de reconocimiento es el necesario para las sentencias de control como la sentencia condicional (**if, else, else_if, end_if...**).

Y por último en el **reconocimiento explícito** se realiza un reconocimiento específico de cada palabra clave. Se reconoce cada palabra independientemente de los demás identificadores y palabras claves. Siempre se utiliza el lexema más largo y en caso de igualdad de longitudes se escoge el componente que se ha especificado primero en la declaración de reglas. El ejemplo del reconocimiento explícito es cuando el componente léxico recibido es un **identificador**.

Ejercicio 3:

a) Resolución según **Algoritmo de Thompson** paso a paso:





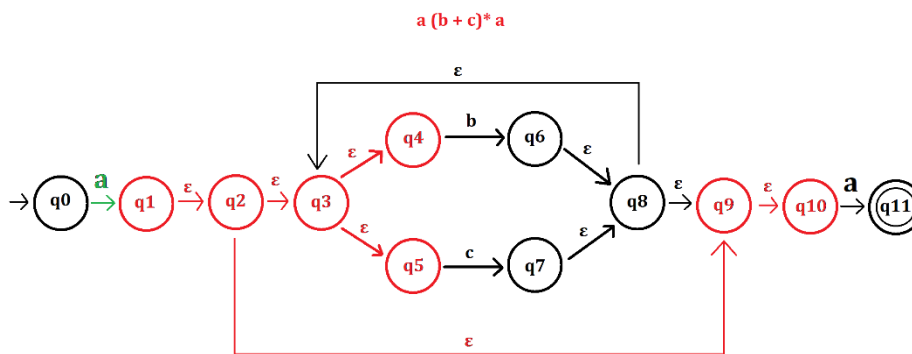
- b) Una vez obtenido el AFN a través del Algoritmo de Thompson, procederemos a obtener el AFD resultante del AFN anterior a través del **Algoritmo de Construcción de Subconjuntos**:

Para obtener el nuevo autómata, lo que tenemos que hacer es, primero, obtener el primer conjunto a través de la función *clausura* $-\epsilon$, que básicamente devuelve el conjunto de estados por los que se pasa si en las transiciones entre estados se encuentra el símbolo épsilon (ϵ). En nuestro caso, como no podemos llegar a otro estado, simplemente el primer conjunto del AFD estará compuesto por el primer estado del AFN:

$$p_0 = \text{clausura} - \epsilon(q_0) = \{q_0\}$$

Una vez obtenido el primer estado del AFD, vamos a conseguir nuevos estados revisando las posibles transiciones del estado p_0 según la declaración de la expresión regular, que en este caso son las letras **a**, **b** y **c**:

$$\begin{aligned} \delta_D(p_0, a) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_0} \delta_N(q, a) \right) = \text{clausura} - \epsilon(\delta_N(q_0, a)) = \\ &= (\text{clausura} - \epsilon(q_1)) = \{q_1, q_2, q_3, q_4, q_5, q_9, q_{10}\} = p_1 \end{aligned}$$



$$\delta_D(p_0, b) = \text{clausura} - \epsilon \left(\bigcup_{q \in p_0} \delta_N(q, b) \right) = \text{clausura} - \epsilon(\delta_N(q_0, b)) =$$

$$= (\text{clausura} - \epsilon(\emptyset)) = \{\emptyset\}$$

$$\delta_D(p_0, c) = \text{clausura} - \epsilon \left(\bigcup_{q \in p_0} \delta_N(q, c) \right) = \text{clausura} - \epsilon(\delta_N(q_0, c)) =$$

$$= (\text{clausura} - \epsilon(\emptyset)) = \{\emptyset\}$$

A partir de aquí continuamos el proceso de obtención de transiciones según los nuevos estados generados. En el siguiente paso vamos a revisar las posibles transiciones del estado p_1 generado en la transición según **a**.

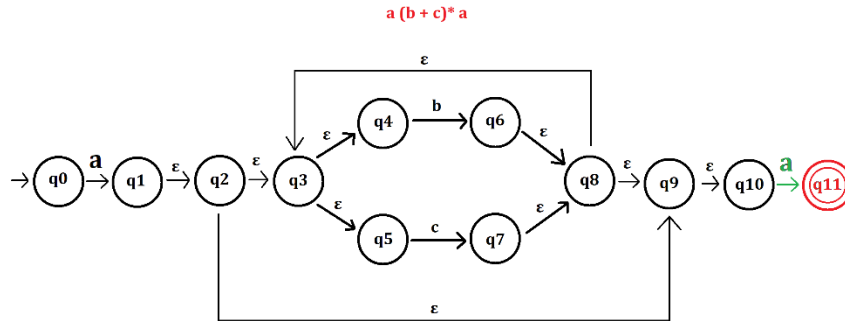
Transiciones de p_1

$$\delta_D(p_1, a) = \text{clausura} - \epsilon \left(\bigcup_{q \in p_1} \delta_N(q, a) \right) =$$

$$= \text{clausura} - \epsilon(\delta_N(q_1, a) \cup \delta_N(q_2, a) \cup \delta_N(q_3, a) \cup \delta_N(q_4, a) \cup \delta_N(q_5, a)$$

$$\cup \delta_N(q_9, a) \cup \delta_N(q_{10}, a)) =$$

$$= (\text{clausura} - \epsilon(q_{11})) = \{q_{11}\} = p_2$$

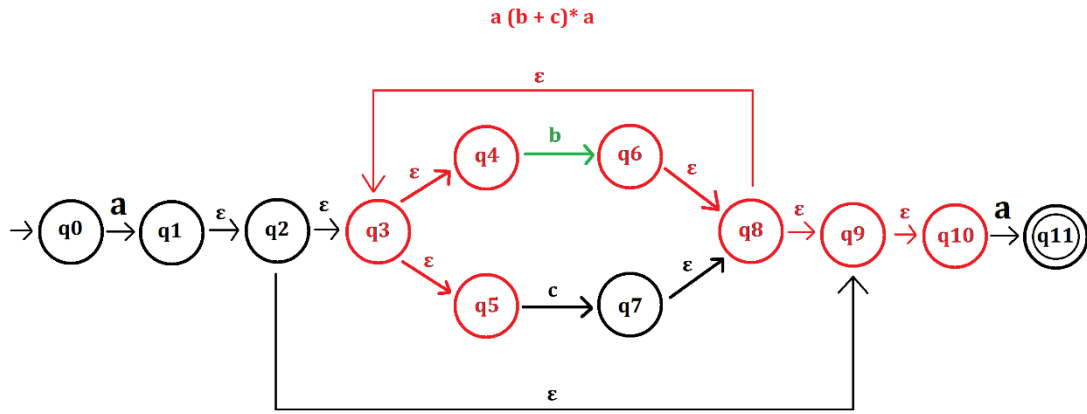


$$\delta_D(p_1, b) = \text{clausura} - \epsilon \left(\bigcup_{q \in p_1} \delta_N(q, b) \right) =$$

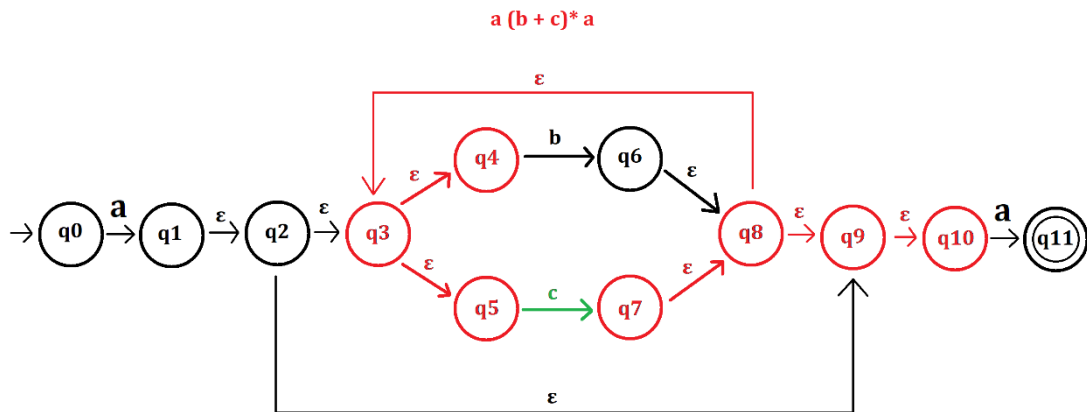
$$= \text{clausura} - \epsilon(\delta_N(q_1, b) \cup \delta_N(q_2, b) \cup \delta_N(q_3, b) \cup \delta_N(q_4, b) \cup \delta_N(q_5, b)$$

$$\cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b)) =$$

$$= (\text{clausura} - \epsilon(q_6)) = \{q_3, q_4, q_5, q_6, q_8, q_9, q_{10}\} = p_3$$



$$\begin{aligned}
 \delta_D(p_1, c) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_1} \delta_N(q, c) \right) = \\
 &= \text{clausura} - \epsilon (\delta_N(q_1, b) \cup \delta_N(q_2, b) \cup \delta_N(q_3, b) \cup \delta_N(q_4, b) \cup \delta_N(q_5, b) \\
 &\quad \cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b)) = \\
 &= (\text{clausura} - \epsilon(q_7)) = \{q_3, q_4, q_5, q_7, q_8, q_9, q_{10}\} = p_4
 \end{aligned}$$



Transiciones de p_2

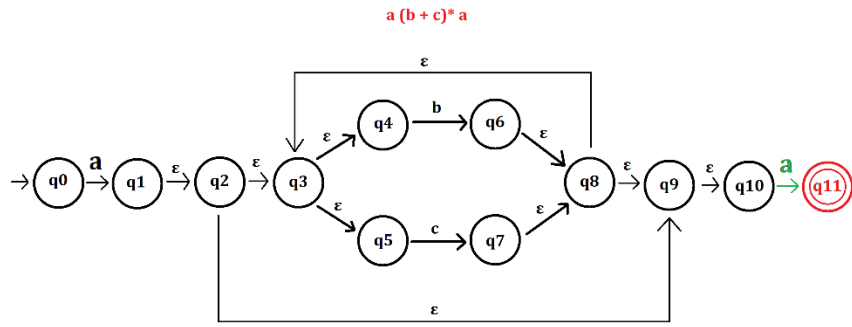
$$\begin{aligned}
 \delta_D(p_2, a) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_2} \delta_N(q, a) \right) = \\
 &= \text{clausura} - \epsilon (\delta_N(q_{11}, a)) = (\text{clausura} - \epsilon(\emptyset)) = \{\emptyset\}
 \end{aligned}$$

$$\begin{aligned}
 \delta_D(p_2, b) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_2} \delta_N(q, b) \right) = \\
 &= \text{clausura} - \epsilon (\delta_N(q_{11}, b)) = (\text{clausura} - \epsilon(\emptyset)) = \{\emptyset\}
 \end{aligned}$$

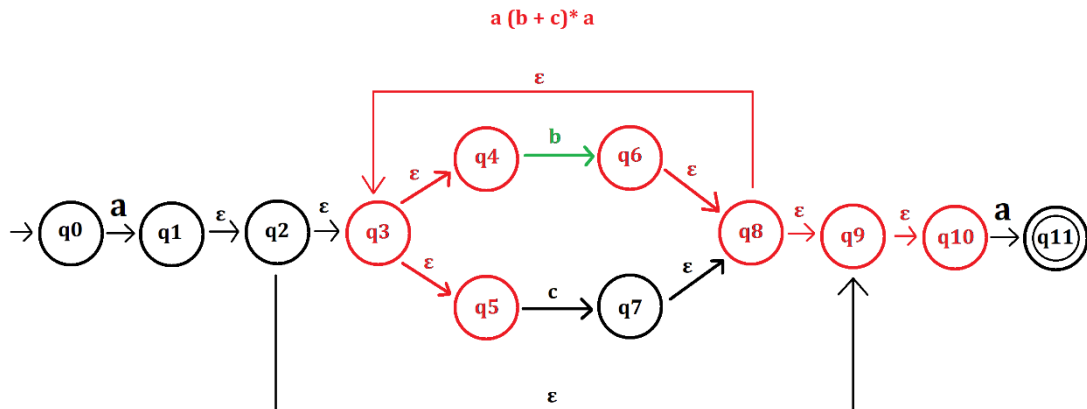
$$\begin{aligned}\delta_D(p_2, c) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_2} \delta_N(q, c) \right) = \\ &= \text{clausura} - \epsilon(\delta_N(q_{11}, c)) = (\text{clausura} - \epsilon(\emptyset)) = \{\emptyset\}\end{aligned}$$

Transiciones de p_3

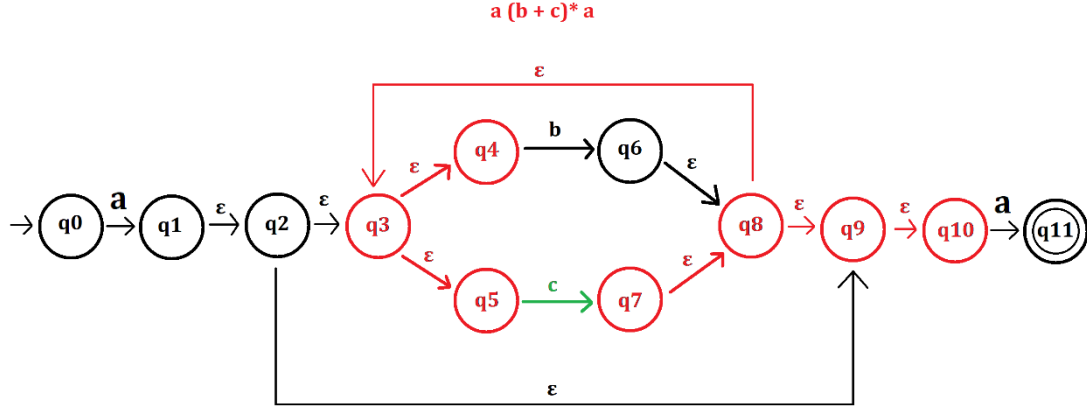
$$\begin{aligned}\delta_D(p_3, a) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_3} \delta_N(q, a) \right) = \\ &= \text{clausura} - \epsilon(\delta_N(q_3, a) \cup \delta_N(q_4, a) \cup \delta_N(q_5, a) \cup \delta_N(q_6, a) \cup \delta_N(q_8, a) \\ &\quad \cup \delta_N(q_9, a) \cup \delta_N(q_{10}, a)) = (\text{clausura} - \epsilon(q_{11})) = \{q_{11}\} = p_2\end{aligned}$$



$$\begin{aligned}\delta_D(p_3, b) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_3} \delta_N(q, b) \right) = \\ &= \text{clausura} - \epsilon(\delta_N(q_3, b) \cup \delta_N(q_4, b) \cup \delta_N(q_5, b) \cup \delta_N(q_6, b) \cup \delta_N(q_8, b) \\ &\quad \cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b)) = \\ &= (\text{clausura} - \epsilon(q_6)) = \{q_3, q_4, q_5, q_6, q_8, q_9, q_{10}\} = p_3\end{aligned}$$

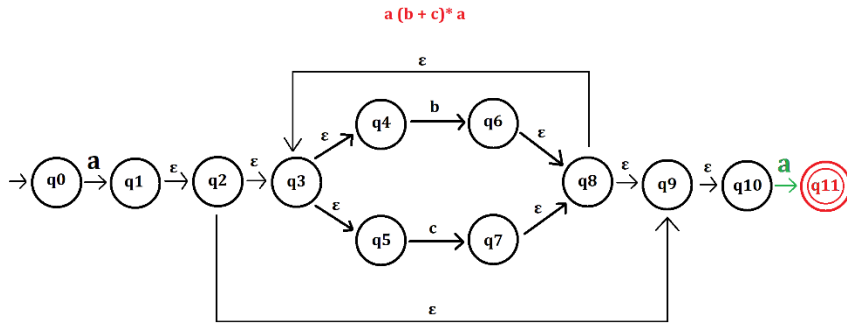


$$\begin{aligned}
\delta_D(p_3, c) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_3} \delta_N(q, c) \right) = \\
&= \text{clausura} - \epsilon (\delta_N(q_3, c) \cup \delta_N(q_4, c) \cup \delta_N(q_5, c) \cup \delta_N(q_6, c) \cup \delta_N(q_8, c) \\
&\quad \cup \delta_N(q_9, c) \cup \delta_N(q_{10}, c)) = \\
&= (\text{clausura} - \epsilon(q_7)) = \{q_3, q_4, q_5, q_7, q_8, q_9, q_{10}\} = p_4
\end{aligned}$$

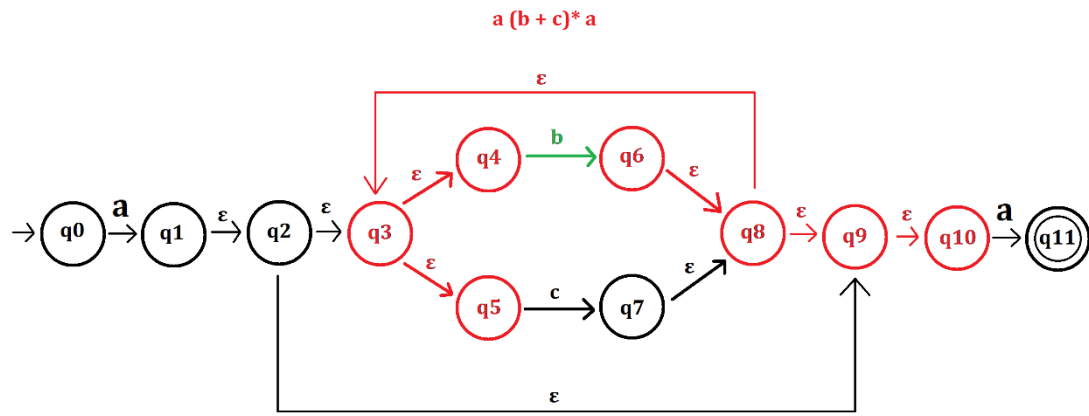


Transiciones de p_4

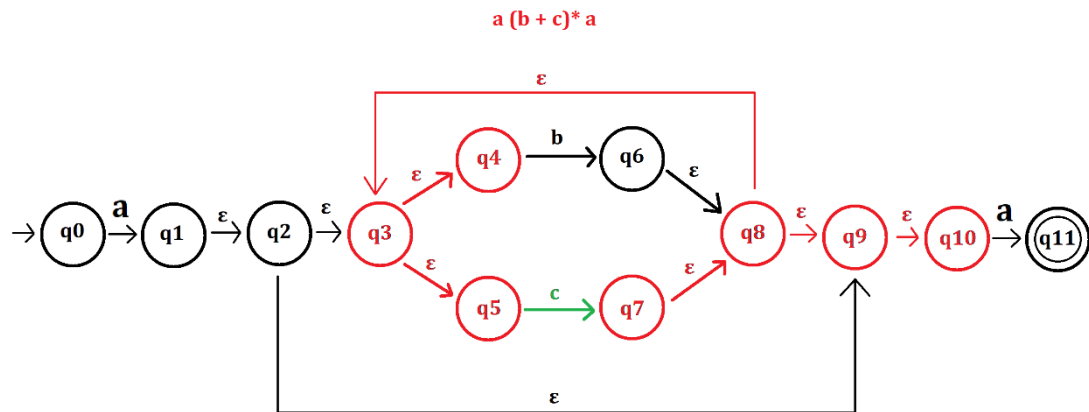
$$\begin{aligned}
\delta_D(p_4, a) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_4} \delta_N(q, a) \right) = \\
&= \text{clausura} - \epsilon (\delta_N(q_3, a) \cup \delta_N(q_4, a) \cup \delta_N(q_5, a) \cup \delta_N(q_7, a) \cup \delta_N(q_8, a) \\
&\quad \cup \delta_N(q_9, a) \cup \delta_N(q_{10}, a)) = (\text{clausura} - \epsilon(q_{11})) = \{q_{11}\} = p_2
\end{aligned}$$



$$\begin{aligned}
\delta_D(p_4, b) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_4} \delta_N(q, b) \right) = \\
&= \text{clausura} - \epsilon (\delta_N(q_3, b) \cup \delta_N(q_4, b) \cup \delta_N(q_5, b) \cup \delta_N(q_6, b) \cup \delta_N(q_8, b) \\
&\quad \cup \delta_N(q_9, b) \cup \delta_N(q_{10}, b)) = \\
&= (\text{clausura} - \epsilon(q_6)) = \{q_3, q_4, q_5, q_6, q_8, q_9, q_{10}\} = p_3
\end{aligned}$$



$$\begin{aligned}\delta_D(p_4, c) &= \text{clausura} - \epsilon \left(\bigcup_{q \in p_4} \delta_N(q, c) \right) = \\ &= \text{clausura} - \epsilon (\delta_N(q_3, c) \cup \delta_N(q_4, c) \cup \delta_N(q_5, c) \cup \delta_N(q_6, c) \cup \delta_N(q_8, c) \\ &\quad \cup \delta_N(q_9, c) \cup \delta_N(q_{10}, c)) = \\ &= (\text{clausura} - \epsilon(q_7)) = \{q_3, q_4, q_5, q_7, q_8, q_9, q_{10}\} = p_4\end{aligned}$$

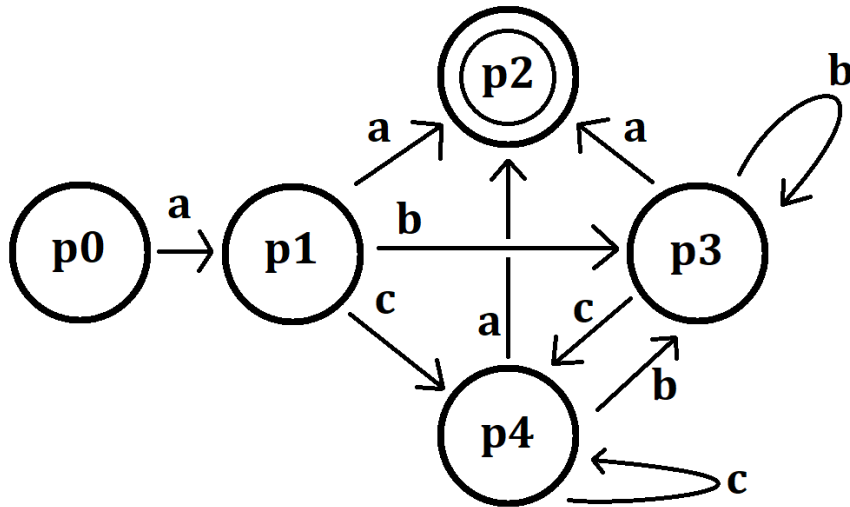


Una vez obtenidos todos los estados del AFD, construimos el autómata según las transiciones. Primero creamos la tabla de transiciones del AFD:

δ	a	b	c
p_0	p_1	-	-
p_1	p_2	p_3	p_4
p_2	-	-	-
p_3	p_2	p_3	p_4
p_4	p_2	p_3	p_4

Ahora posteriormente realizaríamos la representación gráfica del AFD obtenido. Los estados finales del AFD son aquellos los cuales tienen el estado final del AFN. En nuestro caso, el estado final del AFN es el estado

q_{11} , y los estados del AFD que contienen este estado es el estado p_2 por lo que su representación será:



- c) Ahora, procederemos a minimizar el AFD obtenido. Primero, tenemos que dividir los estados en dos grupos: **estados finales** y **estados no finales**. Una vez divididos los estados en esos dos grupos, se va a ir dividiendo los estados del AFD para obtener nuevos estados que sean homogéneos según sus transiciones:

Estados no finales a_0			
δ	a	b	c
p_0	p_1	-	-
p_1	p_2	p_3	p_4
p_3	p_2	p_3	p_4
p_4	p_2	p_3	p_4

Estados finales a_1			
δ	a	b	c
p_2	-	-	-

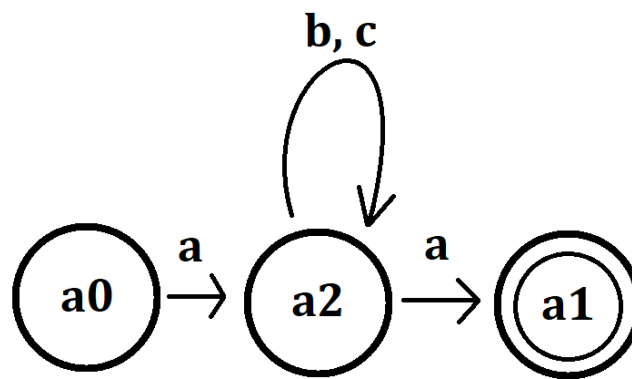
Según las tablas obtenidas, solo nos quedaría dividir la tabla a_0 en dos tablas, obteniendo finalmente tres tablas homogéneas:

Estado a_0			
δ	a	b	c
p_0	a_2	-	-

Estado a_2			
δ	a	b	c
p_1	a_1	a_2	a_2
p_3	a_1	a_2	a_2
p_4	a_1	a_2	a_2

Estado a_1			
δ	a	b	c
p_2	-	-	-

Finalmente, la representación gráfica del AFD mínimo sería el siguiente (la nomenclatura de los estados debe de ser la misma según la creación de las tablas):



d) Ahora, según el AFD mínimo obtenido, comprobemos que reconoce la siguiente sentencia: **a b c a**

$$(a_0, abca) \vdash (a_2, bca) \vdash (a_2, ca) \vdash (a_2, a) \vdash (a_1, \epsilon)$$

Al terminar el proceso en el estado final, significa que el autómata ha reconocido la sentencia.

Ejercicio 4:

a) Partiendo de la gramática inicial, vemos que hay dos reglas con recursividad inmediata por la izquierda (es decir, que en una regla declarada como símbolo NO terminal contiene ese símbolo no terminal a su izquierda):

1. **S** $\rightarrow$ **S** D

5. **L** $\rightarrow$ **L**, id

Para la primera regla creamos las siguientes reglas:

$$S \rightarrow D S'$$

$$S' \rightarrow D S' \mid \varepsilon$$

Y para la segunda regla con recursividad creamos las siguientes reglas:

$$L \rightarrow \text{id } L'$$

$$L' \rightarrow , \text{id } L' \mid \varepsilon$$

El proceso para obtener estas reglas consta de, agregar la nueva regla a la derecha de otras reglas con el mismo símbolo no terminal que no sean las que contengan la recursividad, la regla que contiene la recursividad elimina el símbolo a la izquierda y agrega la nueva regla a la derecha, y por último debe de existir una regla con el nuevo símbolo que llame a la palabra vacía.

Una vez eliminada la recursividad por la izquierda, buscamos una posible factorización por la izquierda, pero al no haber ningún símbolo no terminal con varias reglas que empiecen por el mismo símbolo no podemos hacer la factorización. Por lo que tras realizar los cambios pertinentes la gramática quedaría así:

$P = \{$

1. $S \rightarrow D S'$

2. $S' \rightarrow D S'$

3. $S' \rightarrow \varepsilon$

4. $D \rightarrow L : TV;$

5. $L \rightarrow \text{id } L'$

6. $L' \rightarrow , \text{id } L'$

7. $L' \rightarrow \varepsilon$

8. $T \rightarrow \text{real}$

9. $T \rightarrow \text{entero}$

10. $V \rightarrow \varepsilon$

11. $V \rightarrow = \text{número}$

$\}$

- b) i) Primero vamos a comenzar con la construcción de los conjuntos “Primero” y “Siguiendo”:

Construcción del Conjunto Primero:

La construcción del conjunto primero consta de determinar los símbolos terminales que pueden aparecer a la izquierda de la regla:

Regla 1) $S \rightarrow D S'$ $\text{Primero}(D) - \{\epsilon\} \subseteq \text{Primero}(S)$

Regla 2) $S' \rightarrow D S'$ $\text{Primero}(D) - \{\epsilon\} \subseteq \text{Primero}(S')$

Regla 3) $S' \rightarrow \epsilon$ $\epsilon \in \text{Primero}(S')$

Regla 4) $D \rightarrow L : T V$; $\text{Primero}(L) - \{\epsilon\} \in \text{Primero}(D)$

Regla 5) $L \rightarrow \text{id } L'$ $\text{id} \in \text{Primero}(L)$

Regla 6) $L' \rightarrow , \text{id } L'$ $, , " \in \text{Primero}(L')$

Regla 7) $L' \rightarrow \epsilon$ $\epsilon \in \text{Primero}(L')$

Regla 8) $T \rightarrow \text{real}$ $\text{real} \in \text{Primero}(T)$

Regla 9) $T \rightarrow \text{entero}$ $\text{entero} \in \text{Primero}(T)$

Regla 10) $V \rightarrow \epsilon$ $\epsilon \in \text{Primero}(V)$

Regla 11) $V \rightarrow = \text{número}$ $" = " \in \text{Primero}(V)$

Construcción del Conjunto Siguiendo:

La construcción de este conjunto se acoge a tres reglas determinantes:

1.- $\$ \in \text{Siguiendo}(S)$ donde S es el símbolo inicial de la gramática.

2.- Si $A \rightarrow \alpha B \beta \in P$ entonces $\text{Primero}(\beta) - \{\epsilon\} \subseteq \text{Siguiendo}(B)$

Es decir que, si en una regla hay un símbolo no terminal en medio, el conjunto primero del símbolo de su derecha determina el conjunto Siguiendo de este símbolo.

3.- Si $A \rightarrow \alpha B \in P$, o si $A \rightarrow \alpha B \beta$ y $\epsilon \in \text{Primero}(\beta)$ entonces

$\text{Siguiendo}(A) - \{\epsilon\} \subseteq \text{Siguiendo}(B)$

Es decir, si un símbolo se encuentra a la derecha de una regla o si se encuentra en medio y en el conjunto primero del símbolo de su derecha se encuentra la palabra vacía, el conjunto siguiente del símbolo no terminal

que declara la regla determina el conjunto siguiente del símbolo del que hemos comentado que se encuentra en dentro de la regla.

Regla 1) $S \rightarrow D S'$

Por la regla 1, $\$ \in \text{Siguiente}(S)$.

Por la regla 2, debido a que se determina que $\alpha = \epsilon$;
 $\text{Primero}(S') - \{\epsilon\} \subseteq \text{Siguiente}(D)$.

Por la regla 3.1, $\text{Siguiente}(S) - \{\epsilon\} \subseteq \text{Siguiente}(S')$

Por la regla 3.2, al estar $\epsilon \in \text{Primero}(S')$ significa entonces que
 $\text{Siguiente}(S) - \{\epsilon\} \subseteq \text{Siguiente}(D)$

Regla 2) $S' \rightarrow D S'$

Por la regla 2, obtendríamos una sentencia repetida ya obtenida anteriormente: $\text{Primero}(S') - \{\epsilon\} \subseteq \text{Siguiente}(D)$.

Por la regla 3.1, obtendríamos una regla redundante que ignoraríamos:
 $\text{Siguiente}(S') - \{\epsilon\} \subseteq \text{Siguiente}(S')$

Por la regla 3.2, al estar $\epsilon \in \text{Primero}(S')$ significa entonces que
 $\text{Siguiente}(S') - \{\epsilon\} \subseteq \text{Siguiente}(D)$

Regla 3) $S' \rightarrow \epsilon$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 4) $D \rightarrow L : T V$;

Al estar el símbolo terminal ":" posterior al símbolo no terminal L, significa que ":" $\in \text{Siguiente}(L)$. Lo mismo pasa con el símbolo ";" y V:

";" $\in \text{Siguiente}(V)$.

Por la regla 2, debido a que $\beta = V$; tendríamos que
 $\text{Primero}(V) - \{\epsilon\} \subseteq \text{Siguiente}(T)$.

Por la regla 3.2, debido a que $\epsilon \in \text{Primero}(V)$, significa entonces que
 $\text{Siguiente}(V) - \{\epsilon\} \subseteq \text{Siguiente}(T)$

Regla 5) $L \rightarrow \text{id } L'$

Por la regla 3.1, obtendríamos $\text{Siguiente}(L) - \{\epsilon\} \subseteq \text{Siguiente}(L')$.

Regla 6) $L' \rightarrow , \text{ id } L'$

Por la regla 3.1, obtendríamos la regla superflua que volveríamos a ignorar:
 $\text{Siguiente}(L') - \{\epsilon\} \subseteq \text{Siguiente}(L')$.

Regla 7) $L' \rightarrow \epsilon$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 8) $T \rightarrow \text{real}$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 9) $T \rightarrow \text{entero}$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 10) $V \rightarrow \epsilon$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 11) $V \rightarrow = \text{número}$

No se puede aplicar ninguna regla de construcción del conjunto.

Una vez terminados los procesos de construcción de los conjuntos, obtendríamos la siguiente tabla:

	Primero	Siguiente
S	id	\$
S'	ϵ , id	\$
D	id	id, \$
L	id	“.”
L'	ϵ , “,”	“.”
T	real, entero	“=”, “.”, “,”
V	“=”, ϵ	“.”, “,”

ii) Ahora vamos a proceder a generar la tabla de análisis descendente predictivo según los conjuntos Primero y Siguiente obtenidos. Para la construcción de esta tabla, basta con ir agregando el número de la regla a aplicar en la columna del símbolo terminal que se encuentra en el conjunto primero del símbolo NO terminal. Si hay una regla con la palabra vacía (ϵ), se coloca esa regla en la columna del símbolo terminal **de su conjunto**

Siguiente. Si hay varios símbolos terminales en el conjunto Primero de un símbolo no terminal se aplica la regla correspondiente que genera ese símbolo terminal, como vamos a ver el ejemplo del **símbolo no terminal T**.

	:	;	id	,	real	entero	=	número	\$
S			1						
S'			2						3
D			4						
L			5						
L'	7			6					
T					8	9			
V		10					11		

iii) Ahora vamos a utilizar el método de recuperación de errores “nivel de frase” para completar la tabla. Personalmente recomiendo realizar el siguiente apartado para saber que opciones convienen mejor para rellenar los huecos de la recuperación de errores. Se debe de ampliar la tabla anterior con las filas cuyos símbolos son los terminales. Las celdas a completar son **todas** las celdas vacías **de la parte superior** y en la parte inferior las celdas vacías de las filas de los símbolos terminales que aparecen en alguna regla de producción en un lugar que no es el primero.

	:	;	id	,	real	entero	=	número	\$
S	E1	E2	1	E2	E2	E2	E2	E2	E3
S'	E1	E2	2	E2	E2	E2	E2	E2	3
D	E1	E2	4	E2	E2	E2	E2	E2	E3
L	E1	E2	5	E1	E2	E2	E2	E2	E3
L'	7	E2	E2	6	E5	E5	E2	E2	E3
T	E2	E2	E2	E2	8	9	E2	E2	E3
V	E2	10	E2	E2	E2	E2	11	E4	E3
:	Emp	E2	E2	E2	E2	E2	E2	E5	E3
;	E2	Emp	E2	E2	E2	E2	E2	E2	E3
id	E2	E2	Emp	E2	E2	E2	E2	E2	E3
,				Emp					
real					Emp				
entero						Emp			
=							Emp		
num	E2	E2	E2	E2	E2	E2	E2	Emp	E3
\$	E2	E2	E2	E2	E2	E2	E2	E2	Acep

- **E1** Falta id, insertar id en la entrada.
- **E2** Símbolo inesperado, eliminar símbolo en la entrada.
- **E3** Final inesperado, eliminar símbolo de la pila.
- **E4** Falta “=”. Insertar “=” en entrada
- **E5** Falta “;”. Insertar “;” en entrada.

El error de “final inesperado, eliminar símbolo de la pila” se debe de colocar en la columna del **símbolo terminal \$**, ya que significa que en la entrada se ha llegado al final, pero la pila sigue teniendo elementos residuales, por lo que lo que hay que hacer **es intentar emparejar el símbolo \$ con el de la pila.**

iv) Vamos a proceder a analizar qué es lo que pasaría si se recibe la sentencia *a, , c: entero 1*

Primero transformamos **a y c** en **identificadores** para la entrada, **1** lo determinamos como **número** y agregamos el **símbolo \$** al final de la entrada: **id , , id : entero num \$**

Pila	Entrada	Acción
\$ S	id , , id : entero num \$	1) $S \rightarrow D S'$
\$ S' D	id , , id : entero num \$	4) $D \rightarrow L : T V ;$
\$ S' ; V T : L	id , , id : entero num \$	5) $L \rightarrow id L'$
\$ S' ; V T : L' id	id , , id : entero num \$	Emparejar
\$ S' ; V T : L'	, , id : entero num \$	6) $L' \rightarrow , id L'$
\$ S' ; V T : L' id ,	, , id : entero num \$	Emparejar
\$ S' ; V T : L' id	, id : entero num \$	E2 eliminar símbolo de entrada
\$ S' ; V T : L' id	id : entero num \$	Emparejar
\$ S' ; V T : L'	: entero num \$	7) $L' \rightarrow \epsilon$
\$ S' ; V T :	: entero num \$	Emparejar
\$ S' ; V T	entero num \$	9) $T \rightarrow entero$
\$ S' ; V entero	entero num \$	Emparejar
\$ S' ; V	num \$	E4 Falta “=”. Insertar “=” en entrada.
\$ S' ; V	= num \$	11) $V \rightarrow = num$
\$ S' ; num =	= num \$	Emparejar
\$ S' ; num	num \$	Emparejar
\$ S' ;	\$	E5 Falta “;”. Insertar “;” en entrada.
\$ S' ;	; \$	Emparejar
\$ S'	\$	3) $S' \rightarrow \epsilon$
\$	\$	Aceptar

El resultado final obtenido es: **a , c : entero = 1 ;**

El proceso de análisis descendente predictivo a través de la tabla predictiva consta de una pila y una entrada. En la pila, (de tipo **LIFO: Last In First Out**) comienza con el **símbolo final de la entrada \$** y el **símbolo inicial de la gramática S**. En cada fila se va analizando el token más a la izquierda de la entrada. Según la tabla predictiva, la **pila** determina las **filas** y la **entrada** determina las **columnas**.

Si cae en una celda de la tabla la cual contiene un número, se sustituye el símbolo de la pila por la declaración de la regla específica. Por ejemplo en la siguiente fila al caer en una celda con el número 1, se aplica esa regla:

\$ S	id , , id : entero num \$	1) S → D S'
\$ S' D	id , , id : entero num \$	...

Al tener una pila, colocamos los símbolos del revés ya que se analizaría el símbolo de la derecha de la pila.

Si caemos en la celda de la tabla que dice **Emparejar**, significa que tanto en la entrada como en la pila está el mismo símbolo terminal. Lo que se hace posteriormente a ello es eliminar el símbolo emparejado tanto en la pila como en la entrada:

\$ S' ; V T : L' id	id : entero num \$	Emparejar
\$ S' ; V T : L'	: entero num \$	...

Para el manejo de errores simplemente aplicamos lo que determina esa regla. Si finalmente se cae en la celda que dice **Aceptar**, significa que tanto en la entrada como en la pila SOLO se encuentra el **símbolo final de la entrada \$** y, en consecuencia, se finaliza el análisis.

Aclaración: (Quitando el manejo de errores que aparecen en el ejemplo iv, no estoy muy seguro de lo puesto en la tabla acerca del manejo de errores).

Ejercicio 5:

- a) Previamente a la construcción de los conjuntos primero y siguiente, tenemos que ampliar la gramática para la regla inicial. Este proceso va a consistir en agregar la siguiente regla:

$$1'. S' \rightarrow S$$

$$1. S \rightarrow D$$

Ahora sí, vamos a construir los conjuntos Primero y Siguiente como en el ejercicio anterior según la nueva gramática:

P = {

$$1'. S' \rightarrow S$$

$$1. S \rightarrow SD$$

$$2. S \rightarrow \epsilon$$

$$3. D \rightarrow \text{struct id } \{ L \};$$

$$4. L \rightarrow LI$$

$$5. L \rightarrow I$$

$$6. I \rightarrow T \text{ id};$$

$$7. T \rightarrow \text{float}$$

$$8. T \rightarrow \text{int}$$

}

Construcción del Conjunto Primero:

La construcción del conjunto primero consta de determinar los símbolos terminales que pueden aparecer a la izquierda de la regla:

$$\text{Regla } 1') S' \rightarrow S \quad \text{Primero}(S) - \{\epsilon\} \subseteq \text{Primero}(S')$$

$$\text{Regla } 1) S \rightarrow D \quad \text{Primero}(D) - \{\epsilon\} \subseteq \text{Primero}(S)$$

$$\text{Regla } 2) S \rightarrow \epsilon \quad \epsilon \in \text{Primero}(S)$$

$$\text{Regla } 3) D \rightarrow \text{struct id } \{ L \}; \quad \text{struct} \in \text{Primero}(D)$$

$$\text{Regla } 4) L \rightarrow LI \quad \text{Primero}(L) - \{\epsilon\} \subseteq \text{Primero}(L) \text{ (redundancia)}$$

$$\text{Regla } 5) L \rightarrow I \quad \text{Primero}(I) - \{\epsilon\} \subseteq \text{Primero}(L)$$

Regla 6) $I \rightarrow T \text{ id}; \quad \text{Primero}(T) - \{\epsilon\} \subseteq \text{Primero}(I)$

Regla 7) $T \rightarrow \text{float} \quad \text{float} \in \text{Primero}(T)$

Regla 8) $T \rightarrow \text{int} \quad \text{int} \in \text{Primero}(T)$

Construcción del Conjunto Siguiente:

Una vez explicadas las reglas a aplicar en el ejercicio 4, procedemos a obtener el conjunto:

Regla 1') $S' \rightarrow S$

Por la regla 1, $\$ \in \text{Siguiente}(S')$.

Por la regla 3.1, $\text{Siguiente}(S') - \{\epsilon\} \subseteq \text{Siguiente}(S)$

Regla 1) $S \rightarrow S D$

Por la regla 2, significa que $\text{Primero}(D) - \{\epsilon\} \subseteq \text{Siguiente}(S)$.

Por la regla 3.1, obtendríamos una regla redundante que ignoraríamos:
 $\text{Siguiente}(S) - \{\epsilon\} \subseteq \text{Siguiente}(D)$

Regla 2) $S \rightarrow \epsilon$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 3) $D \rightarrow \text{struct id } \{ L \};$

Al estar el símbolo terminal “}” posterior al símbolo no terminal L, significa que “}” $\in \text{Siguiente}(L)$.

Regla 4) $L \rightarrow L I$

Por la regla 2, significa que $\text{Primero}(I) - \{\epsilon\} \subseteq \text{Siguiente}(L)$.

Por la regla 3.1, obtendríamos $\text{Siguiente}(L) - \{\epsilon\} \subseteq \text{Siguiente}(I)$.

Regla 5) $L \rightarrow I$

Por la regla 3.1, obtendríamos la regla ya mencionada previamente:
 $\text{Siguiente}(L) - \{\epsilon\} \subseteq \text{Siguiente}(I)$.

Regla 6) $I \rightarrow T \text{ id};$

Al estar el símbolo terminal “id” posterior al símbolo no terminal T, significa que “id” $\in \text{Siguiente}(T)$.

Regla 7) $T \rightarrow \text{float}$

No se puede aplicar ninguna regla de construcción del conjunto.

Regla 8) $T \rightarrow \text{int}$

No se puede aplicar ninguna regla de construcción del conjunto.

Una vez terminados los procesos de construcción de los conjuntos, obtendríamos la siguiente tabla:

	Primero	Siguiente
S'	ϵ , struct	\$
S	ϵ , struct	\$, struct
D	struct	\$, struct
L	float, int	"}", float, int
I	float, int	"}", float, int
T	float, int	id

- b) Vamos a proceder con la construcción de la colección canónica de LR(0)-elementos. Este proceso comienza con la función clausura de la regla principal. Se define un punto que se coloca antes de la definición de la regla. La función clausura devuelve la regla definida previamente más las reglas de **los símbolos no terminales** que se encuentran justo a la derecha del punto en reglas anteriores. Para las reglas con la palabra vacía solo se implementa un punto:

$$I_0 = \text{clausura}(\{S' \rightarrow \cdot S\}) = \{S' \rightarrow \cdot S, S \rightarrow \cdot S D, S \rightarrow \cdot\}$$

Ahora procederemos a ver las posibles transiciones del estado I_0 según los símbolos que se encuentran posterior al punto en cada uno de los elementos del conjunto. Para ello se avanza el punto y se hace la función clausura de cada uno:

Transiciones de I_0

$$\begin{aligned} Ir_a(I_0, S) &= \text{clausura}(\{S' \rightarrow S \cdot, S \rightarrow S \cdot D, D \rightarrow \cdot \text{struct id } \{L\};\}) = \\ &= \{S' \rightarrow S \cdot, S \rightarrow S \cdot D, D \rightarrow \cdot \text{struct id } \{L\};\} = I_1 \end{aligned}$$

Una vez obtenidas las transiciones del primer estado, procedemos a repetir este proceso para cada uno de los estados definidos hasta que no se generen nuevos estados:

Transiciones de I_1

$$Ir_a(I_1, \mathbf{D}) = \text{clausura}(\{S \rightarrow S \mathbf{D} \cdot\}) = \{S \rightarrow S \mathbf{D} \cdot\} = I_2$$

$$Ir_a(I_1, \mathbf{struct}) = \text{clausura}(\{D \rightarrow \mathbf{struct} \cdot id \{L\};\}) = \{D \rightarrow \mathbf{struct} \cdot id \{L\};\} = I_3$$

Transiciones de I_2

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

Transiciones de I_3

$$Ir_a(I_3, \mathbf{id}) = \text{clausura}(\{D \rightarrow struct \mathbf{id} \cdot \{L\};\}) = \{D \rightarrow struct \mathbf{id} \cdot \{L\};\} = I_4$$

Transiciones de I_4

$$\begin{aligned} Ir_a(I_4, \{\}) &= \text{clausura}(\{D \rightarrow struct id \{ \cdot L \};\}) = \\ &= \{D \rightarrow struct id \{ \cdot L \};, L \rightarrow \cdot L I, L \rightarrow \cdot \mathbf{I}, \mathbf{I} \rightarrow \cdot \mathbf{T} id;, \mathbf{T} \rightarrow \cdot float, \mathbf{T} \rightarrow \cdot int\} = \\ &= I_5 \end{aligned}$$

Aquí al llamar de nuevo a L ($L \rightarrow \cdot L I$) no sería necesario implementar dos veces las reglas $\{L \rightarrow \cdot L I, L \rightarrow \cdot I\}$ ya que con incluirlo solo una vez es suficiente.

Transiciones de I_5

$$\begin{aligned} Ir_a(I_5, \mathbf{L}) &= \text{clausura}(\{D \rightarrow struct id \{ \mathbf{L} \cdot \};, L \rightarrow \mathbf{L} \cdot \mathbf{I}\}) = \\ &= \{D \rightarrow struct id \{ \mathbf{L} \cdot \};, L \rightarrow \mathbf{L} \cdot \mathbf{I}, \mathbf{I} \rightarrow \cdot \mathbf{T} id;, \mathbf{T} \rightarrow \cdot float, \mathbf{T} \rightarrow \cdot int\} = I_6 \end{aligned}$$

$$Ir_a(I_5, \mathbf{I}) = \text{clausura}(\{L \rightarrow \mathbf{I} \cdot\}) = \{L \rightarrow \mathbf{I} \cdot\} = I_7$$

$$Ir_a(I_5, \mathbf{T}) = \text{clausura}(\{I \rightarrow \mathbf{T} \cdot id\}) = \{I \rightarrow \mathbf{T} \cdot id\} = I_8$$

$$Ir_a(I_5, \mathbf{float}) = \text{clausura}(\{T \rightarrow \mathbf{float} \cdot\}) = \{T \rightarrow \mathbf{float} \cdot\} = I_9$$

$$Ir_a(I_5, \mathbf{int}) = \text{clausura}(\{T \rightarrow \mathbf{int} \cdot\}) = \{T \rightarrow \mathbf{int} \cdot\} = I_{10}$$

Transiciones de I_6

$$Ir_a(I_6, \{\}) = \text{clausura}(\{D \rightarrow struct id \{L\} \cdot\}) = \{D \rightarrow struct id \{L\} \cdot\} = I_{11}$$

$$Ir_a(I_6, \mathbf{I}) = \text{clausura}(\{L \rightarrow L \mathbf{I} \cdot\}) = \{L \rightarrow L \mathbf{I} \cdot\} = I_{12}$$

$$Ir_a(I_6, \mathbf{T}) = \text{clausura}(\{I \rightarrow \mathbf{T} \cdot id\}) = \{I \rightarrow \mathbf{T} \cdot id\} = I_8$$

$$Ir_a(I_6, \mathbf{float}) = \text{clausura}(\{T \rightarrow \mathbf{float} \cdot\}) = \{T \rightarrow \mathbf{float} \cdot\} = I_9$$

$$Ir_a(I_6, \mathbf{int}) = \text{clausura}(\{T \rightarrow \mathbf{int} \cdot\}) = \{T \rightarrow \mathbf{int} \cdot\} = I_{10}$$

Transiciones de I_7

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

Transiciones de I_8

$$Ir_a(I_8, id) = clausura(\{I \rightarrow T \text{ id } \cdot ;\}) = \{I \rightarrow T \text{ id } \cdot ;\} = I_{13}$$

Transiciones de I_9

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

Transiciones de I_{10}

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

Transiciones de I_{11}

$$Ir_a(I_{11}, ;) = clausura(\{D \rightarrow struct \text{ id } \{L\} ; \cdot\}) = \{D \rightarrow struct \text{ id } \{L\} ; \cdot\} = I_{14}$$

Transiciones de I_{12}

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

Transiciones de I_{13}

$$Ir_a(I_{13}, ;) = clausura(\{I \rightarrow T \text{ id } ; \cdot\}) = \{I \rightarrow T \text{ id } ; \cdot\} = I_{15}$$

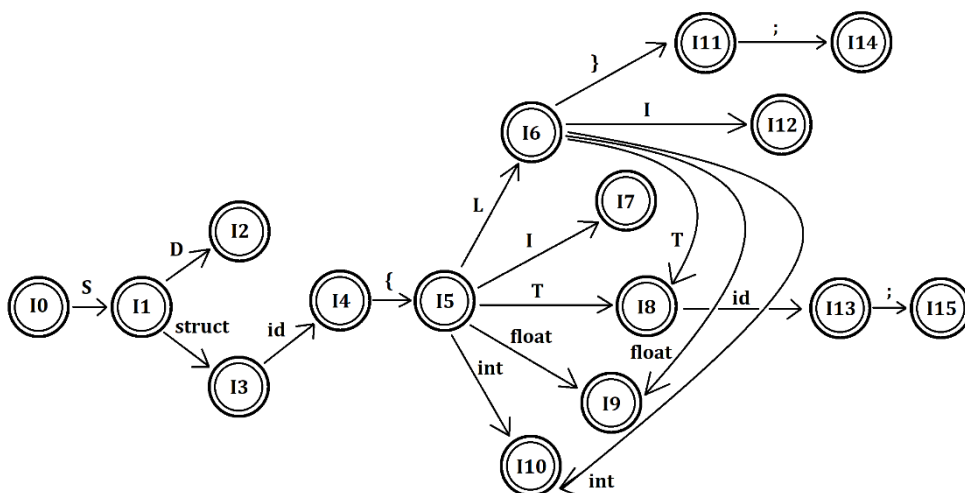
Transiciones de I_{14}

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

Transiciones de I_{15}

No tiene transiciones porque no hay ningún símbolo a la derecha del punto.

- c) Una vez terminadas todas las transiciones de los estados y no obtener ningún nuevo estado, procedemos a representar gráficamente el autómata. El proceso consta de ir viendo las transiciones entre estados I_x según qué símbolos se utilizan:



En este autómata generado por el proceso anterior, todos los estados son estados finales.

- d) Para la construcción de la tabla de análisis sintáctico SLR, vamos a intentar realizar el siguiente proceso:

Primero, vamos a visualizar la parte del autómata, y según las transiciones que refleja la imagen, **pondremos a que estado deberá de desplazarse**. En la **parte de ir_a** (donde se encuentran los símbolos no terminales) simplemente hará falta colocar el número del estado al que transiciona, y en la **parte de acción** (donde se encuentran los símbolos terminales) pondremos la letra **d** junto al número del estado al que se desplaza. Luego **para la parte de reducción** haremos uso del **Conjunto Siguierte**, ya que, para poder reducir una expresión según una regla gramatical, tendremos que ver si en un estado hay una expresión con el punto al final (por ejemplo, esta expresión: $\{ L \rightarrow L I \cdot \}$) y una vez aquí aplicaremos la regla gramatical correspondiente **(4)** en la columna de los símbolos terminales que tiene en su conjunto siguiente **(“”, float, int)**:

	Primero	Siguierte
S'	ϵ , struct	\$
S	ϵ , struct	\$, struct
D	struct	\$, struct
L	float, int	“”, float, int
I	float, int	“”, float, int
T	float, int	id

También hay que recordar que, en la parte de reducción, si el punto está a la derecha o al final de la regla inicial, que en este ejemplo la regla inicial es $\{ S' \rightarrow S \cdot \}$, en la columna de los símbolos terminales que aparecen en su conjunto Siguierte (\$), estará la regla **Aceptar**, la cual se corresponde con la regla que determina si lo que se ha reconocido es correcto según la gramática del enunciado:

	acción								ir_a				
	struct	id	{	float	int	}	;	\$	S	D	L	I	T
I_0	r2							r2	1				
I_1	d3							Aceptar		2			
I_2	r2							r1					
I_3		d4											
I_4			d5										
I_5				d9	d10						6	7	8
I_6				d9	d10	d11						12	8
I_7				r5	r5	r5							
I_8		d13											
I_9		r7											
I_{10}		r8											
I_{11}							d14						
I_{12}				r4	r4	r4							
I_{13}							d15						
I_{14}	r3							r3					
I_{15}				r6	r6	r6							

e) Ahora intentaremos implementar el método de error a nivel de frase:

	acción								ir_a				
	struct	id	{	float	int	}	;	\$	S	D	L	I	T
I_0	r2	E2	E2	E2	E2	E2	E2	r2	1				
I_1	d3	E2	E2	E2	E2	E2	E2	Aceptar		2			
I_2	r2	E2	E2	E2	E2	E2	E2	r1					
I_3	E2	d4	E2	E2	E2	E2	E2	E2					
I_4	E2	E2	d5	E4	E2	E2	E2	E2					
I_5	E2	E2	E2	d9	d10	E2	E2	E2			6	7	8
I_6	E2	E2	E2	d9	d10	d11	E2	E2				12	8
I_7	E2	E2	E2	r5	r5	r5	E2	E2					
I_8	E2	d13	E2	E2	E2	E2	E2	E2					
I_9	E2	r7	E2	E2	E2	E2	E2	E2					
I_{10}	E2	r8	E2	E2	E2	E2	E2	E2					
I_{11}	E5	E5	E5	E5	E5	E5	d14	E5					
I_{12}	E2	E2	E2	r4	r4	r4	E2	E2					
I_{13}	E5	E5	E5	E5	E5	E5	d15	E5					
I_{14}	r3	E2	E2	E2	E2	E2	E2	E2					
I_{15}	E2	E2	E2	r6	r6	r6	E2	E2					

- **E1** Falta id, insertar id en la entrada.
- **E2** Símbolo inesperado, eliminar símbolo en la entrada.
- **E3** Final inesperado, eliminar símbolo de la pila.
- **E4** Falta "{". Insertar "{" en entrada
- **E5** Falta ";". Insertar ";" en entrada.

(Quitando el manejo de errores que aparecen en el ejemplo f) no estoy muy seguro de lo puesto en la tabla del manejo de errores).

f) Finalmente, una vez terminada la tabla de análisis sintáctico ascendente SLR procedemos a analizar la sentencia: **struct PUNTO float ; y }**

Antes de comenzar con el análisis cambiaremos los **nombres de variables por identificadores** y añadiremos el **símbolo \$** al final de la entrada: **struct id float ; id } \$**

El proceso del análisis ascendente se explica posterior a esta tabla de análisis:

Pila	Entrada	Acción
0	struct id float ; id } \$	r2 S → ε
0 S 1	struct id float ; id } \$	d3
0 S 1 struct 3	id float ; id } \$	d4
0 S 1 struct 3 id 4	float ; id } \$	E4 → Falta "{"
0 S 1 struct 3 id 4	{ float ; id } \$	d5
0 S 1 struct 3 id 4 { 5	float ; id } \$	d9
0 S 1 struct 3 id 4 { 5 float 9	; id } \$	E2 → Símbolo inesperado, eliminar símbolo en la entrada.
0 S 1 struct 3 id 4 { 5 float 9	id } \$	r7 T → float
0 S 1 struct 3 id 4 { 5 T 8	id } \$	d13
0 S 1 struct 3 id 4 { 5 T 8 id 13	} \$	E5 → Falta ";"
0 S 1 struct 3 id 4 { 5 T 8 id 13	; } \$	d15
0 S 1 struct 3 id 4 { 5 T 8 id 13 ; 15	} \$	r6 I → T id ;
0 S 1 struct 3 id 4 { 5 I 7	} \$	r5 L → I
0 S 1 struct 3 id 4 { 5 L 6	} \$	d11
0 S 1 struct 3 id 4 { 5 L 6 } 11	\$	E5 → Falta ";"
0 S 1 struct 3 id 4 { 5 L 6 } 11	; \$	d14
0 S 1 struct 3 id 4 { 5 L 6 } 11 ; 14	\$	r3 D → struct id { L } ;
0 S 1 D 2	\$	r1 S → S D
0 S 1	\$	Aceptar

El resultado final detectado según el análisis ascendente SLR es el siguiente: **struct PUNTO { float y ; } ;**

Explicación del proceso:

El proceso de análisis ascendente es un poco más complejo que el de análisis descendente. Partimos de la pila en el **estado 0**, y vamos recibiendo los componentes léxicos de la entrada, el cual analizamos el que está más a la izquierda. El número que se encuentra más a la derecha de la pila, define el estado actual en el que se encuentra el autómata, por ejemplo, esta expresión define que estamos en el **estado 5** del autómata:

0 S 1 struct 3 id 4 { 5	float ; id } \$	d9
--------------------------------	------------------------	----

En la tabla superior el estado actual siempre se encuentra **en negrita**.

Cuando el autómata cae en una celda de la tabla que define un **desplazamiento**, lo que hacemos es mover el token analizado en la fila anterior a la derecha de la pila, y posterior a ello colocamos el número del estado al que nos hemos desplazado a la derecha del token:

0 S 1	struct id float ; id } \$	d3
0 S 1 struct 3	id float ; id } \$	...

Por último, cuando el autómata cae en una celda de la tabla que define una **reducción**, tendremos que obtener la regla que se cita (en este ejemplo r6 llama a la regla 6 de la gramática **I → T id ;**). Una vez en este punto, **se revisa que existan todos los componentes de la regla en la pila** (significa que la tabla está bien construida hasta ese momento) y prestamos atención **al número que se sitúa previamente a la regla** (en este caso, como la regla es **T id ;** nos fijamos en el número anterior a esta expresión que es **5**).

Sustituimos en la siguiente fila la expresión (en nuestro ejemplo sustituiríamos **T id ;** por el símbolo no terminal **I**) y, según el número previo que determinaría en este paso intermedio el estado actual, vemos a donde transicionar si se recibe el símbolo sustituido (en nuestro ejemplo **I**. En la tabla en el estado 5 si recibe **I** transicionamos al estado 7 según la parte de la tabla **ir_a**) y **se coloca este nuevo estado a la derecha de la pila**.

0 S 1 struct 3 id 4 { 5 T 8 id 13 ; 15	} \$	r6 I → T id ;
0 S 1 struct 3 id 4 { 5 I 7	} \$	...

Para el manejo de errores de nuevo se vuelve a aplicar lo que nos dice ese error, y si cae en la celda que dice **Aceptar** significa que se ha finalizado el análisis correctamente.

Ejercicio 6:

El enunciado nos pide que creemos una gramática que permita esto:

$$L = \{a^i b^{2j} c^j d^{2i} \mid i > 0, j > 1\}$$

Una de las posibles gramáticas que englobe esta expresión es la siguiente:

P= {

1. $S \rightarrow A B$
2. $A \rightarrow a A d d$
3. $A \rightarrow a d d$
4. $B \rightarrow b b B c$
5. $B \rightarrow b b b b c c$

}

Para llegar a esta expresión tendremos que hacer un proceso:

- 1- Juntar las letras o tokens que cuyo número de repeticiones dependen entre sí en una regla, y separar las que no dependan entre sí. En este ejemplo las letras **a y d** dependen entre sí, pero no dependen ambas de **b y c**, que estas dos sí que dependen entre sí por lo que los dividimos en la **regla A** (que controla **a y d**) y la **regla B** (que controla **b y c**).
- 2- En la regla A, se pide que mínimo exista una letra a, por lo que en ambas reglas de A se pone una simple **a**, pero para **d** que debe de ser el doble de a se escriben dos d (**a A d d**). Se escriben dos reglas en la que una tiene la letra A en medio **para que se puedan crear más letras a y d** ya que **tiene que ser mayor a 0 pero NO igual a 1**, y se escribe otra regla sin el símbolo no terminal para que no quede en un bucle infinito intentando crear la expresión.
- 3- En la regla B se pide que mínimo existan dos **c** y cuatro **b**. Por lo que la regla B que no se llama a sí misma debe de tener esta cantidad de símbolos terminales, pero **esto no hace falta para la regla que se llama a sí misma** ya que lo único que importa es que **haya el doble de letras b que de c**.